Du terrain nu au rendu HR 3D

Comment une carte d'Artouste se construit et se dessine

Olivier Booklage

version 0.45.0.714, le 9 octobre 2026

Quatre étapes : partir de zéro, le rendu LR, le rendu HR, le rendu HR 3D. Les fonctions sont citées à chaque étape pour retrouver le code.

1 Fabriquer une carte

Une carte se fabrique hors du jeu, avec `python3 tools/fetch_terrain.py <zone>`. Ce script en appelle quatre autres :

- `tools/terrain/zones/<zone>.py` décrit la zone : emprise en lon/lat, taille de la grille d'altitudes, hauteur de l'orthophoto, point de départ, hélisurfaces ;
- `tools/terrain/relief.py` demande les altitudes à l'API altimétrie de l'IGN (RGE ALTI, https://data.geopf.fr/altimetrie/1.0/calcul/alti/rest/elevation.json) et les écrit dans `heightmap.png`, un PNG 16 bits étalé de 0 à `elev_max` ;
- `tools/terrain/ortho.py` télécharge l'orthophoto BD ORTHO par le WMS de la Géoplateforme (https://data.geopf.fr/wms-r/wms, couche `ORTHOIMAGERY.ORTHOPHOTOS`), comble le blanc en bordure de couverture (`fill_nodata()`), donne une couleur unie à la mer (`flatten_sea()`) et écrit `ortho.jpg` ;
- `tools/terrain/outputs.py` et `meta.py` écrivent `terrain.txt` (emprise, maillage, altitude maximale). Le script pose aussi `landmarks.txt` (lieux remarquables) et `helipads.txt`.

Le dossier `assets/terrain/<carte>/` obtenu est la carte livrée : l'état LR. Toutes les données viennent de l'IGN (https://geoservices.ign.fr/), sous Licence Ouverte Etalab 2.0.

2 Le rendu LR : un maillage et une photo

Au chargement, `Terrain::Terrain()` (`src/render/Terrain.cpp`) lit `terrain.txt`, `heightmap.png` et `ortho.jpg`. Puis `Terrain::construireMaillage()` (`src/render/terrain/TerrainMaillage.cpp`) échantillonne la heightmap sous un plafond de sommets (`relief_sommets_max`) : sur une carte de 18 km, la maille tombe à 15-20 m. Le maillage est construit une fois, `Terrain::draw()` le dessine à chaque image.

Le shader (`assets/shaders/terrain.frag`) drape `ortho.jpg` dessus et y mêle un grain rocheux selon la pente. La physique lit le même relief par `Terrain::heightAt()` (interpolation bilinéaire de la heightmap).

Résultat : correct de loin, flou au ras du sol (une dizaine de mètres par pixel), anguleux sur les crêtes.

3 Le rendu HR : des tuiles d'image fines

Le gestionnaire de cartes (touche C, puis Entrée) fabrique des tuiles d'orthophoto à 0,25 m par pixel. `Fabrique::lancer()` (`src/app/cartes/FabriqueTuiles.cpp`) télécharge l'image bloc par bloc (même WMS, même couche que le socle), la découpe en tuiles de 512 px, les compresse en BC7 et les écrit en `.dds` (`render::dds::ecrire()`), avec un index par niveau de finesse. Le script `tools/terrain/fetch_tuiles.py` fait pareil en ligne de commande ; les deux produisent le même jeu et une fabrication commencée par l'un se reprend par l'autre.

Au chargement, `ouvrirNiveaux()` (`src/render/tuiles/Pyramide.cpp`) retrouve le jeu. Une `Fenetre` (`src/render/tuiles/Fenetre.hpp`) par niveau garde une texture de tuiles autour de la caméra : `Fenetre::suivre()` la recentre à chaque image et charge les tuiles qui entrent, un masque dit au shader lesquelles sont prêtes. Dans `terrain.frag`, `poserTuiles()` pose le niveau large puis le niveau serré sur l'orthophoto d'ensemble, avec un fondu par distance (`reglerNiveau()` dans `src/app/render/ApplicationRenderTerrain.cpp`).

Résultat : le sol est net en vol bas. Le relief, lui, reste celui du maillage LR.

4 Le rendu HR 3D : la fenêtre de relief

La méthode est propre à Artouste, construite pour ce jeu, mesure par mesure. Ses quatre choix : le relief fin est un *écart* ajouté à la carte, pas une surface de plus (carte + détail = laser, par construction) ; le pas des tuiles s'emboîte dans la maille du terrain ; la finesse ne dépend que de la distance, pour que les grilles lisent la même surface à leur jonction ; la physique recalcule la formule du shader, donc l'appareil se pose sur ce que le pilote voit.

La touche L du gestionnaire fabrique le jeu de relief. `Fabrique::lancerRelief()` puis `Fabrique::boucleRelief()` (`src/app/cartes/fabrique/FabriqueReliefBoucle.cpp`) téléchargent le MNT LiDAR HD de l'IGN, le modèle du sol nu (couche `IGNF_LIDAR-HD_MNT_ELEVATION.ELEVATIONGRIDCOVERAGE.WGS84G` du même WMS ; le MNS contient les toits et les cimes, on se poserait dessus), en binaire brut (`demanderAltitudes()`). Les trous et les points aberrants sont bouchés avec le relief de la carte (`ReliefCarte::altitude()`), puis tout est écrit en tuiles `.r16` sur 16 bits (`ecrireBlocRelief()`, `encoderTuile()`). Le pas vient de `grilleRelief()` : la maille de la carte divisée par un multiple de 4, par axe. Un pas libre ferait voir la frontière de la fenêtre en vol. Le script `tools/terrain/fetch_relief.py` fait pareil en ligne de commande (le format des tuiles est décrit dans son en-tête et dans `tools/terrain/relief_tuiles.py`).

Au chargement, `cheminJeuDeRelief()` trouve le dossier `<carte>.relief` et `FenetreRelief::ouvrir()` (`src/render/relief/`) alloue une texture d'altitudes avec ses niveaux de réduction, et deux grilles sans données : le noyau (pas fin, environ 2 m) et son anneau (pas quadruple). `FenetreRelief::suivre()` recentre le tout sur l'appareil à chaque image.

Au dessin (`Application::renderTerrainAndBuildings()`), l'anneau est tiré d'abord, le noyau par-dessus (profondeur `GL_LEQUAL` : à égalité, le dernier tiré gagne). Le pochoir écarte le maillage d'ensemble de l'emprise de la fenêtre. Dans `terrain.vert`, chaque sommet retrouve sa position depuis son numéro (`gl_VertexID`), lit l'altitude de la carte (`hauteurCarte()`) et l'écart du laser (`detailFin()`, au niveau de réduction donné par `finesseDetail()`), puis fond cet écart vers zéro au bord de la fenêtre (`poidsFenetre()`) : `hauteurFenetre()` = carte + poids × détail. À la frontière, la surface rejoint donc exactement le maillage d'ensemble. Les normales sont recalculées par différences finies (`normaleDe()`), et `terrain.frag` écarte les points hors de l'emprise de la carte.

La physique suit la même formule : `FenetreRelief::detailEn()` recalcule côté processeur le même écart et le même poids, `Terrain::heightAt()` l'ajoute à l'altitude de la carte. On se pose sur ce qu'on voit ; c'est un invariant du moteur.

5 Modélisation

En un point p du sol, à distance d de l'œil de la fenêtre :

$$h(p) = c(p) + t(d)\,\big[\,T_\ell(p) - c(p)\,\big]$$

$c(p)$ est l'altitude de la carte (bilinéaire sur `heightmap.png`), $T_\ell(p)$ l'altitude laser lue au niveau de réduction ℓ, $t(d)$ le poids du fondu. Le détail est l'écart $T - c$, jamais une altitude : quand t tombe à zéro, la surface *est* la carte. Aucune marche possible à la frontière.

La finesse ne dépend que de la distance :

$$\ell(d) = \min\Big(\max\big(\log_2 \tfrac{d}{D},\, 0\big),\, L\Big) \qquad \text{avec} \qquad L = \left\lfloor \log_2 \tfrac{\Delta_{\text{carte}}}{\Delta_{\text{relief}}} \right\rfloor$$

Pleine finesse jusqu'à D (environ 226 m sur dax), puis résolution divisée par deux à chaque doublement de distance, plafonnée à L, le niveau qui lisse le laser à la maille de la carte. Comme ℓ ne dépend que de d, le noyau et l'anneau lisent la même surface là où ils se rejoignent.

Le pas des tuiles s'emboîte dans la maille m de la carte :

$$\Delta_{\text{relief}} = \frac{m}{4k}, \qquad k \in \mathbb{N}^* \text{ choisi pour approcher 2 m}$$

Sur dax : $m = 14{,}156 \times 21{,}763$ m donne $14{,}156/8 = 1{,}7695$ m et $21{,}763/12 = 1{,}8136$ m. 8 et 12 sont multiples de 4 : le noyau et l'anneau (qui dessine à 4 fois ce pas) tombent tous deux sur les nœuds du maillage.

Chaque tuile de 512×512 points quantifie ses altitudes sur 16 bits, bornes propres à la tuile :

$$a = a_{\min} + \frac{n}{65535}\,e, \qquad n \in \{0, \dots, 65535\}, \qquad \varepsilon_{\max} = \frac{e}{131070}$$

où e est l'étendue d'altitude de la tuile : 1 cm d'erreur sur 1300 m de dénivelé, 0,08 mm sur une tuile de plaine de 10 m d'étendue. Mesuré contre la chaîne Python de référence : 0,59 mm d'écart maximal sur 2,4 millions de points.

Les normales sont des différences finies centrales, au pas δ suivant la finesse :

$$\vec{n} = \text{normalize}\left(\frac{h(x-\delta,z) - h(x+\delta,z)}{2\delta},\ 1,\ \frac{h(x,z-\delta) - h(x,z+\delta)}{2\delta}\right)$$

Pourquoi la double précision : autour de 43 degrés, deux `float` consécutifs sont séparés de

$$\text{ulp}(43) = 43 \times 2^{-23} \approx 5{,}1 \times 10^{-6} \text{ degré} \approx 40 \text{ cm au sol.}$$

En simple précision, le relief sortait décalé d'un demi-mètre sur les versants. En double, le même écart tombe au dixième de micron.

6 Voir la mécanique en vol

La clé `relief_debug 1` de `assets/config.txt` trace les frontières de la fenêtre : contour du noyau en magenta, contour de l'anneau en cyan, cercles du fondu en orange et jaune, légende dans le HUD 4 coins (touche `H`). Le journal du lancement donne le reste : maille LR, finesse des tuiles HR, pas des grilles, et la ligne `emboîtement dans la carte : oui`.

7 Les difficultés rencontrées et les solutions

Des silhouettes qui bougeaient à la frontière de la fenêtre. Avec un pas de tuiles de 2,00 m partout, la fenêtre redessinait la surface du maillage au lieu de la reproduire : les crêtes changeaient de forme en entrant. Solution : le pas n'est plus libre, `grilleRelief()` le cale sur la maille de la carte divisée par un multiple de 4, par axe (`pasEmboite()`).

Des pics qui surgissaient à l'approche des crêtes. En réalité un peigne de lames verticales, une par cellule d'anneau, à partir du cercle de départ du fondu. Les données, la loi de finesse, la couture de la texture et les indices des grilles ont été mis hors de cause un par un, mesure après mesure. La cause : le noyau, tiré en premier, réservait ses pixels au pochoir ; sur une arête vue en rasant, ses triangles de 2 m se projettent en lamelles de moins d'un pixel, les pixels non couverts restaient libres et l'anneau y montrait sa propre corde, plus basse. Chaque grille seule dessinait la crête proprement ; le défaut venait de leur arbitrage. Solution : inverser l'ordre, l'anneau d'abord sur toute son emprise, le noyau par-dessus en `GL_LEQUAL` (à égalité, le dernier tiré gagne). Coût nul. La physique n'a jamais été touchée : `detailEn()` lit le champ continu, c'est pour ça que l'appareil s'enfonçait dans les dents dessinées.

Des changements de couleur en approchant une montagne. Deux causes. Au bord de carte, la fenêtre débordait de l'emprise et ses textures en répétition y dessinaient l'orthophoto et le relief de l'autre côté de la carte : corrigé par un écart au fragment hors emprise (`terrain.frag`). Et un versant pouvait passer clair, sombre, puis clair pendant l'approche : les normales sont recalculées à un pas qui suit la finesse (`pasNormale()`), l'éclairage dépend donc de la distance. L'essentiel a disparu avec le peigne ; le reste est surveillé.

Un décalage de 80 cm du relief sur les versants. Les bornes géographiques de la fabrique étaient en simple précision (voir la section des formules). Leur passage en double (`CalageCarte`) a aussi corrigé le même décalage sur les tuiles d'orthophoto.

Des points laser qui ne mesuraient pas le sol. Certains retours plongent de plusieurs centaines de mètres sous le relief (jusqu'à −796 m sur un versant d'Ossau). La fabrique écarte tout point à plus de 300 m sous la carte (`CHUTE_ABERRANTE_M`) et bouche trous et aberrations avec le relief d'ensemble (`ReliefCarte::altitude()`).

À retenir de l'enquête du peigne : n'afficher qu'une grille à la fois pour isoler le fautif, compter les traversées de silhouette le long d'une ligne plutôt que des écarts entre pixels, couper le MSAA avant toute sonde qui encode des mesures dans les couleurs.

8 Sources de données

— Géoplateforme IGN, service WMS raster : https://data.geopf.fr/wms-r/wms (orthophotos BD ORTHO et MNT LiDAR HD).
— API altimétrie IGN (RGE ALTI) : https://data.geopf.fr/altimetrie/1.0/calcul/alti/rest/elevation.json.
— Portail des géoservices : https://geoservices.ign.fr/.
— Licence Ouverte Etalab 2.0 : https://www.data.gouv.fr/pages/legal/licences/etalab-2.0.